Frontier Models as Weight Editors for Gemma 4 E4B

Ozgun Genc
ozgungenc@gmail.com

A gradient-free weight edit cuts a coding agent's escape-code errors from 15 % to 3 % in live runs, and shows what such fixes must also preserve

Extended version of our Kaggle Writeup for the Gemma 4 Developer Agent Competition, with the result tables, the first round, the editors' decisions and the retry checks. Protocols, run records and audits are in the repository.

Abstract

We study whether frontier models can change a coding agent's behaviour by editing its weights directly. The target is one defect of Gemma 4 E4B: when it edits a file, it often copies escape codes such as `\n` into the text it means to replace. Editors could read the weights and send at most 25 forward-pass requests, but could not compute gradients. In a test fixed in advance on 69 held-out repositories, three submitted edits cut these escape artefacts by 23, 53 and 75 %, beyond size-matched random controls, but each also lost 3 to 11 of 213 previously working edit calls in a draw, where our preservation rule allowed 2. Served as a LoRA adapter on 4-bit E4B in vLLM, one selected edit cut the defect in live agent runs on 148 new repositories from 15 % to 3 % of edit calls, and from 7 % to 1 % with thinking on, in two replications planned before their runs, and it ran in the competition's own scorer. Gradient training on the same calls removed more artefacts and failed the same rule. We detected no difference in solved tasks: the harness already repairs many escaped copies. Re-scored afterwards without that repair, fewer calls work, but two of the three edits lose none of them and raise edit acceptance by 4 and 7 points. Most of the base model's misses are retries after an earlier miss, and two checks fixed in advance found little room for an edit there. We release the library, a new task set and the replay test set.

1 Introduction

Gemma 4 E4B is a small open model that works as a coding agent in the Gemma 4 Developer Agent Competition's harness. To change a file, the agent calls an edit tool with the *old text*, copied exactly from the file, and the new text. The model sees files in an escaped form, where a line break is the two characters `\n` and a quote is `\"`, and it often copies that form into the old text (Figure 1). We call each such leftover an *escape artefact*.

Failed edits are frequent: in 412 agent runs on four training repositories, 405 of 1,191 edit calls failed because the old text was not in the file. The harness already softens the escape defect. When an old text is not found, a fallback unescapes both the old text and its replacement and tries again; it rescued 108 of 614 accepted edits. So we did not expect fewer escape artefacts to solve many more tasks. We chose the defect because it is frequent, countable and specific, which makes it a clean test of a narrower question: can a frontier model deliberately change one behaviour of a small model by writing its weights, without breaking the calls that already work?

Preprint.

(a) The file on disk

```
def total(items):
    return sum(item["price"] for item in items)
```

(b) What the model reads: the same text inside the tool result, written with escape codes

```
{"content": "def total(items):\n    return sum(item[\"price\"] for item in items)\n"}
```

(c) The old text the model writes when it edits this file

Unedited model

```
def total(items):\n    return sum(item[\"price\"] for item in items)
```

✗ not in the file; only a fallback that removes escapes rescues it

Edited model

```
def total(items):
    return sum(item["price"] for item in items)
```

✓ found in the file as written

escape code: \n stands for a line break, \" for a quote

Figure 1: The defect, in a synthetic example: the unedited model copies escape codes into the old text; the edited model writes the real characters.

Frontier models have recently been shown to write changes to neural-network weights directly, an approach called weight-space programming [1]. We test the idea on one behaviour of a software-engineering agent, along three separate questions (Figure 2): does the behaviour change, are working calls preserved, and does the agent solve more tasks? We contribute a controlled experiment with frontier-model weight editors, held-out repositories and size-matched random controls; tests of one selected edit in live, quantized serving and against gradient training on the same calls; and a library, a task set and a replay set for repeating such tests. The experiment tests particular edits under fixed budgets; it does not map the limits of weight editing.

2 Method

2.1 Editors

Each second-round editor worked alone in a sandbox, with the weights and recorded calls from four training repositories (fastapi, rich, requests and httpx) but no access to the test data (Figure 3a). It could read any weight and send forward-pass requests that return outputs or activations. The edits are *gradient-free*: backpropagation and iterative optimisers were not allowed, but closed-form fits on activations, such as least squares, were. No editor's scripts or transcripts contain a gradient or optimiser call. Each editor could submit one edit or none (Table 1).

Table 1: Second-round editor arms. "Informed" arms could read the earlier editors' work; each was registered separately.

Arm	Model	Forward-pass budget	Started from	Submitted
Opus	Claude Opus 5.5	25	base model	an edit
Fable	Claude Fable 5.1	25	base model	an edit
Astra	GPT-6 Astra	10	base model	nothing
Informed Opus	Claude Opus 5.5	10	base model, earlier editors' records	an edit

Arm	Model	Forward-pass budget	Started from	Submitted
Informed Fable 5.2	Claude Fable 5.2	10	base model, earlier editors' records	nothing (withdrew)
Repair Fable 5.2	Claude Fable 5.2	10	a gradient-trained adapter (Section 3.2)	nothing (withdrew)

Each submission is a set of low-rank changes: a weight matrix W becomes W + BA, where B and A are two thin matrices. This form is a LoRA adapter [2], which the competition's vLLM server can load. Fable's edit is a rank-2 change in the last layer's feed-forward output: one part detects activations just before an escaped punctuation token, and the other shifts token scores toward the raw form, both fitted with linear solves. Opus instead turned 40 little-used last-layer neurons into detectors, gated by a rank-1 flag in layer 40. These descriptions are the editors' own.

2.2 How we judged an edit

We judged each frozen edit by replay (Figure 3b). For a recorded edit call, the model regenerates only the old text from the exact original prompt, and the harness's matching code checks it against the file. On held-out calls, the replay reproduced 95 of 103 recorded failures and all 213 successes.

The test was fixed in advance [3]: 320 calls from 101 tasks in 69 held-out repositories, each generated in two *draws*, one greedy and one at temperature 0.6. Each edit faced the base model and two random controls with the same location, rank and size but a random direction, and its per-task artefact rate had to fall against all three (one-sided sign tests, Holm-corrected across candidates). Three guards limited side effects:

- *acceptance*, the share of old texts found, averaged over tasks, could fall by at most 1 point;
- among recorded calls that had succeeded and that the base model's replay also matched (213 per draw), which we call *working calls*, at most 2 could be lost in each draw (negative flips [12]); these are accepted old-text matches, not correct bug fixes;
- *compile failures* among accepted calls could rise by at most 2 points, measured on a replay that regenerates both the old and the new text, because the fallback changes both. This full-call form replaced the registered one before any edit ran on the test calls; Table 2 gives both.

The round-2 rule was committed locally before the first forward pass and pushed to GitHub while the search was under way. The live replications and the training comparison each had a plan with an expected effect, frozen and hashed in our log before their runs and committed to git afterwards. We chose the target after counting artefacts in the base model's test calls.

2.3 Task sets and the live metric

The replay calls come from agent runs on two held-out sets, OOD-43 and OOD-B, and the first live run used their 115 tasks. For the planned replications we built OOD-C: 186 tasks from 148 repositories, filtered from SWE-rebench [4, 5], packaged to run offline in the competition harness, and checked by script to share no repository with training, the earlier sets or the replay calls. Only 53 OOD-C tasks meet our full gradability rule (the reference fix passes twice, the empty patch fails, and the reference fix passes again under Python 3.12), and we count solved tasks on these 53. So OOD-C mainly measures what the agent writes. In live runs we count edit calls whose old *or* new text carries an artefact and leave out calls whose file state could not be rebuilt; counting the old text alone gives the same picture.

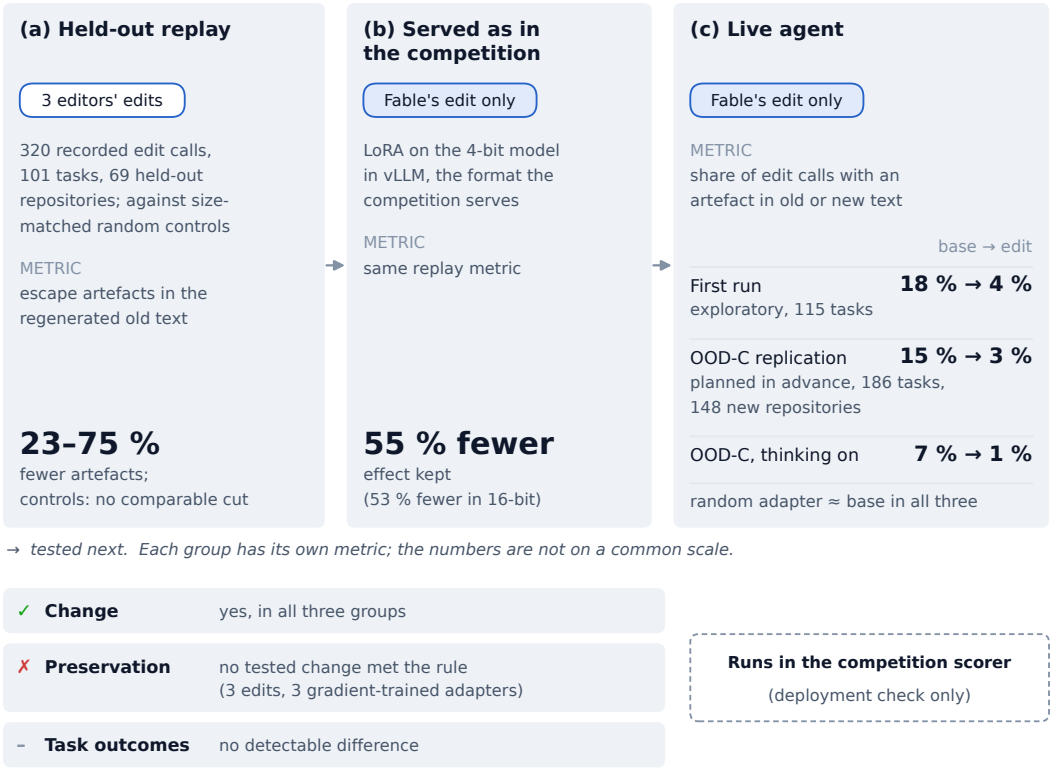


Figure 2: Evidence map: three groups of tests, each with its own metric and edits; arrows mean "tested next". Below: the three outcomes and a deployment check.

3 Results

3.1 Targeted change and preservation

All three second-round edits reduced the defect, and every one lost more working calls than the rule allowed (Figure 4, Table 2). Of 640 replay generations, 100 of the base model's carried an artefact; Opus's edit left 77, Fable's 47 and informed Opus's 25: cuts of 23, 53 and 75 %, each with $p \leq 0.011$ against the base and both of its controls. The six random controls left 99 to 102. But the three edits lost up to 3, 5 and 11 of the 213 working calls in a draw, against a limit of 2, and Fable's (narrowly) and informed Opus's edits also failed the acceptance guard.

Table 2: Second-round results: every edit passed the artefact test and failed at least one guard (320 calls, two draws each).

Edit (requests)	Escape artefacts of 640 ↓ (base 100)	Tasks fewer / more	Largest p of 3	Acceptance change, points (≥ -1)	Working calls lost per draw (≤ 2)	Compile failures, registered / full call (≤ 11.0 / 9.9 %)	Passed
Opus 5.5 (25)	77	9 / 0	0.011	+0.4	2, 3	12.5 / 8.4 %	no
Fable 5.1 (25)	47	20 / 0	9.5×10^{-7}	−1.01	5, 5	15.3 / 9.1 %	no
Informed Opus 5.5 (10)	25	25 / 1	4.0×10^{-7}	−4.2	11, 11	15.9 / 8.4 %	no

Edit (requests)	Escape artefacts of 640 ↓ (base 100)	Tasks fewer / more	Largest p of 3	Acceptance change, points (≥ −1)	Working calls lost per draw (≤ 2)	Compile failures, registered / full call (≤ 11.0 / 9.9 %)	Passed
Six random controls	99–102						

A cleaner-looking copy can be less reliable than an escaped one (Figure 5). When the base model wrote an escaped old text, the fallback often repaired it and the call succeeded. Writing raw text there gains nothing, and the raw copy is sometimes wrong, for example with a closing brace dropped. Of the 84 escaped copies that the fallback had rescued for the base model, the three edits moved 24, 45 and 67 off the fallback, most of them to exact copies, and 5, 12 and 24 of those failed. Average acceptance can hide such losses when other calls improve, so the rule counts them separately.

The same generations show what that repair hides. Scored again with the harness's own matching code but its unescape fallback switched off (post hoc; nothing is regenerated), the base model's accepted calls fall from 443 to 359 of 640, and its working calls per draw from 213 to 177 and 174. Opus's and Fable's edits then lose none of these, pass all three guards and raise acceptance by 4.1 and 7.3 points (better in 10 and 17 tasks, worse in none; beyond both controls). Informed Opus's edit gains 9.3 points but misses the compile guard by 0.1 point. No agent has run without the fallback.

The editors' own decisions add context; they are single runs, not a ranking of models. Informed Opus named the lost-call guard as its main risk and submitted anyway: on its own data its edit had lost up to 5 calls per draw, against 11 here, while its artefact cut transferred (82 % there, 75 % here). Astra submitted nothing, because its best candidate showed no effect on tasks it had set aside; it also found, on its own, the flaw in the registered compile guard that led us to amend it. Both Fable 5.2 runs found effects but withdrew after measuring side effects themselves. The informed run's best candidate cut artefacts by two thirds on fresh search calls but broke legitimate escapes, such as `\n\n` in new code. The repair run found that the gradient-trained adapter it started from cut artefacts but ended quoted spans early, and that no repair kept as many accepted-and-compiling edits as the base model; on this harness, it argued, the fallback already accepts the artefact calls. Gated afterwards off the record, the informed run's withdrawn candidate left 43 artefacts against 100 but failed all three guards (acceptance −1.27 points, 3 working calls lost per draw, compile failures +3.7 points), close to its editor's own estimates.

3.2 Comparison with gradient training

Editing by reasoning would be most useful if it needed less data than training. In a comparison fixed in advance, we trained LoRA adapters with gradients to write the correct old text on calls from the training repositories. The matched arm used the 39 labelled calls among those Fable had read, at Fable's location, rank and size. Two larger arms trained all linear layers, on the same 39 calls and on all 411 labelled calls; these two are not size-matched. Only the 39-call arms tuned their learning rate, on validation calls from the training repositories.

Training removed more artefacts and failed the same rule (Figure 6, Table 3). At Fable's location the trained adapter was lower than Fable's edit in 13 tasks and higher in 1 ($p = 0.002$). Every arm lost more than 2 working calls per draw; the 411-call arm passed every other guard. It was also cheap: a 39-call run took about 1 h and the 411-call run 7.3 h on the same Mac, while Fable's 25 forward-pass requests took 10.6 h. On this target the authored edit showed no data advantage over training, and the loss of working calls was not specific to gradient-free editing. Nor must removing escape artefacts break calls: the all-layer arms removed more while losing fewer, and without the fallback (Section 3.1) the adapters gain 9 to 16 points of acceptance, though each still misses a guard.

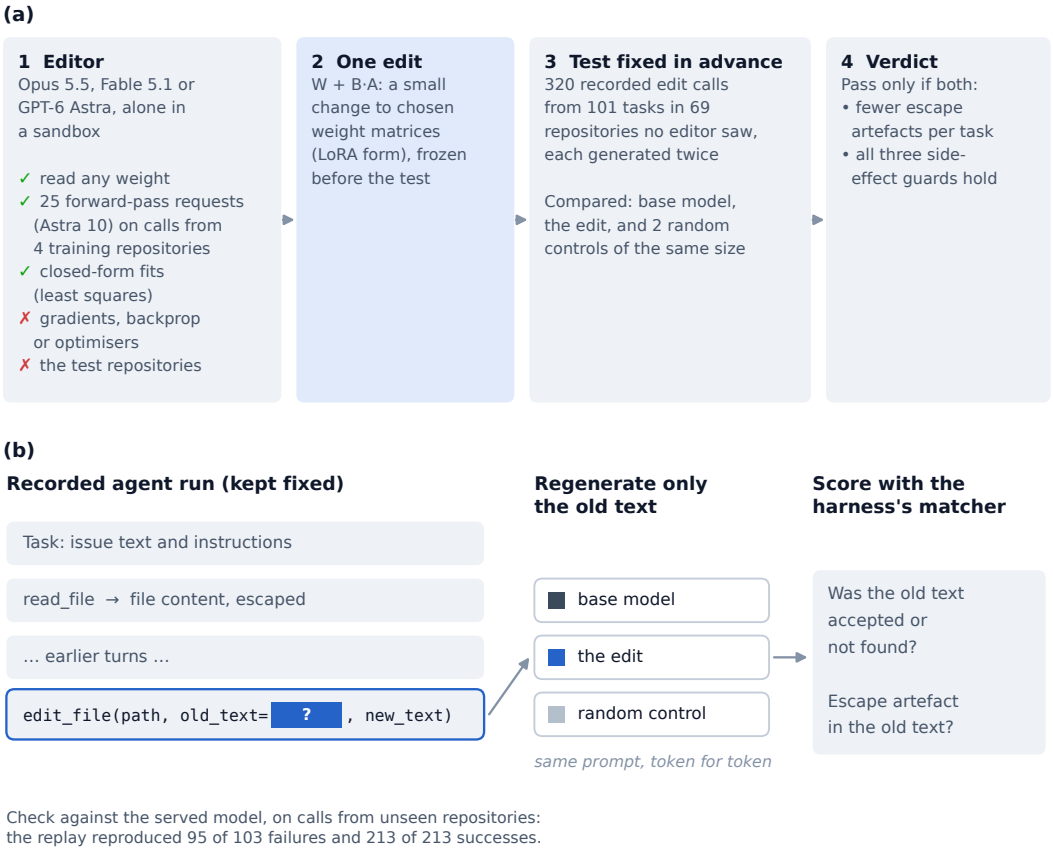


Figure 3: (a) What an editor could and could not do. (b) The replay regenerates only the old text; everything before it stays fixed.

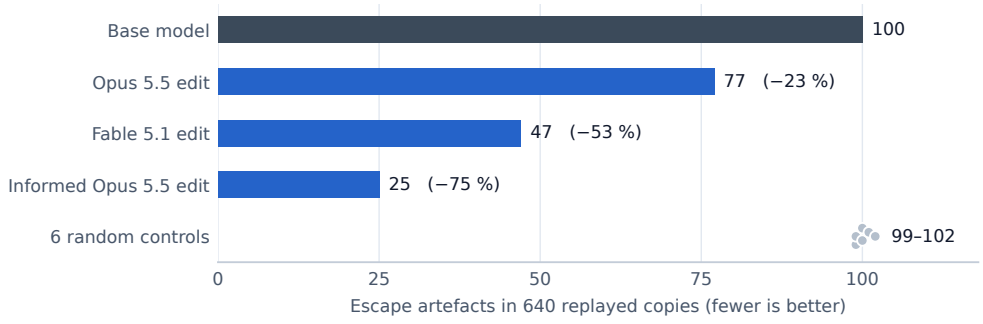


Figure 4: Every edit cut escape artefacts beyond its own size-matched random controls; the edits differ in size.

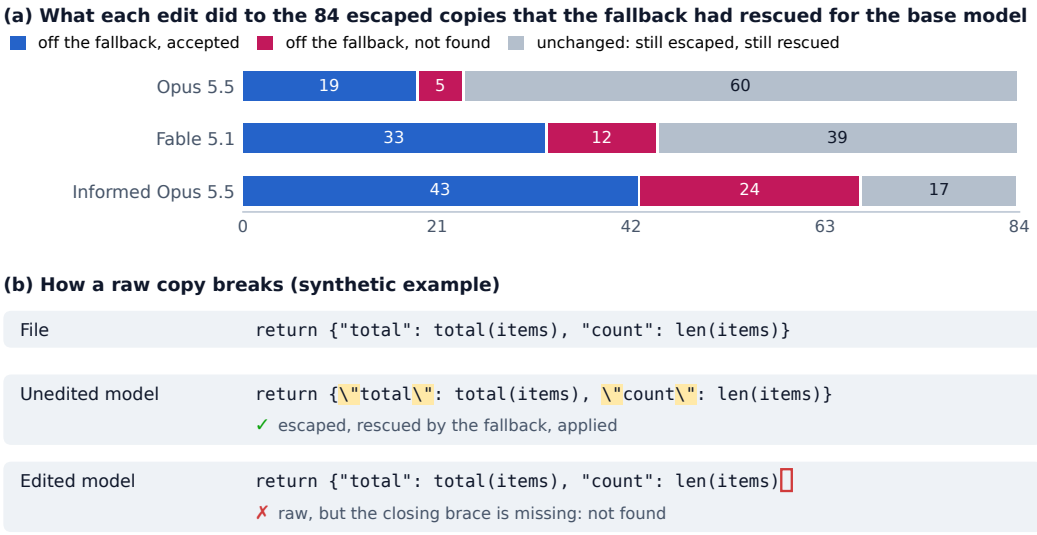


Figure 5: (a) What each edit did to the escaped copies that the fallback had rescued for the base model: some left the fallback and were still accepted, some were no longer found. (b) How a raw copy breaks (synthetic example).

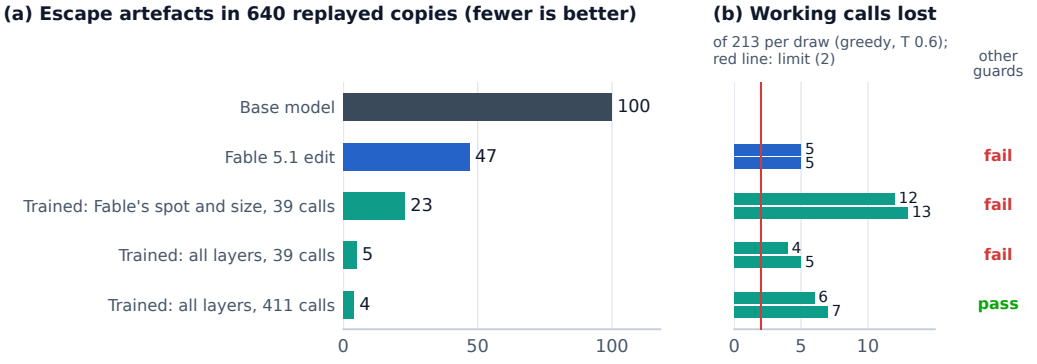


Figure 6: Gradient training on the same calls removed more artefacts than Fable's edit, and every arm lost more than 2 working calls per draw.

Table 3: Gradient-trained adapters against Fable's edit on the replay test (the same rule as Table 2).

Arm	Escape artefacts of 640 ↓ (base 100)	Acceptance change, points (≥ -1)	Working calls lost per draw (≤ 2)	Compile failures, full call (≤ 9.9 %)	Passed
Fable 5.1 edit (25 requests)	47	−1.0	5, 5	9.1 %	no
Trained at Fable's location and size, 39 calls	23	−3.6	12, 13	10.3 %	no
Trained on all layers, 39 calls	5	+1.1	4, 5	10.7 %	no

Arm	Escape artefacts of 640 ↓ (base 100)	Acceptance change, points (≥ -1)	Working calls lost per draw (≤ 2)	Compile failures, full call (≤ 9.9 %)	Passed
Trained on all layers, 411 calls	4	+0.3	6, 7	5.2 %	no

3.3 Transfer to the running agent

We chose Fable's edit after seeing the replay results, so this step is exploratory. Served as a LoRA adapter on 4-bit (W4A16) E4B in vLLM, as the competition serves it, the edit kept its effect: artefacts fell from 102 to 46, close to its 16-bit result, while a random adapter left 100. Its cost survived too: 7 and 11 working calls lost per draw (5 and 5 in 16-bit; none for the random adapter). In a rehearsal on the scorer's hardware vLLM served all 223 requests through the adapter without errors, and agents with this edit ran in the competition's scorer, scoring 0.05 with thinking off and on, like our comparable agents without it (0.05 to 0.08). The board shows only that the edit runs there, not what it changes.

In live agent runs the edit cut the defect again, on new repositories, under a plan fixed in advance (Figure 7, Table 4). A first, exploratory run on the 115 OOD-43 and OOD-B tasks gave 4 % of counted edit calls with an artefact, against 18 % for the base and 20 % for a random adapter. Because those repositories had supplied the replay calls, we froze the edit, the analysis, the pass rule and an expected effect, and repeated the three arms on OOD-C. Artefacts fell from 15 % of counted edit calls for the base to 3 % for the edit, with 16 % for the random adapter. Per task, the edit was lower than the base in 34 tasks and higher in 4 ($p = 3 \times 10^{-7}$), and lower than the random adapter in 27 and higher in 7 ($p = 4 \times 10^{-4}$); a repository-level bootstrap also supported the cut. The live gain does not lift the replay's preservation failure.

The agent we submit thinks before it acts; the runs above did not. A second planned replication on the same OOD-C tasks, with thinking on, gave 7 % for the base, 1 % for the edit and 8 % for the random adapter. The edit was lower than the base in 16 tasks and higher in 4 ($p = 0.006$), and passed against the random adapter too ($p = 2 \times 10^{-4}$). The two replications ran in separate sessions, so their base rates do not measure what thinking does. In all three runs the edit made the agent lean less on the fallback, which rescued 3 to 6 % of its accepted edits against 9 to 19 % for the base.

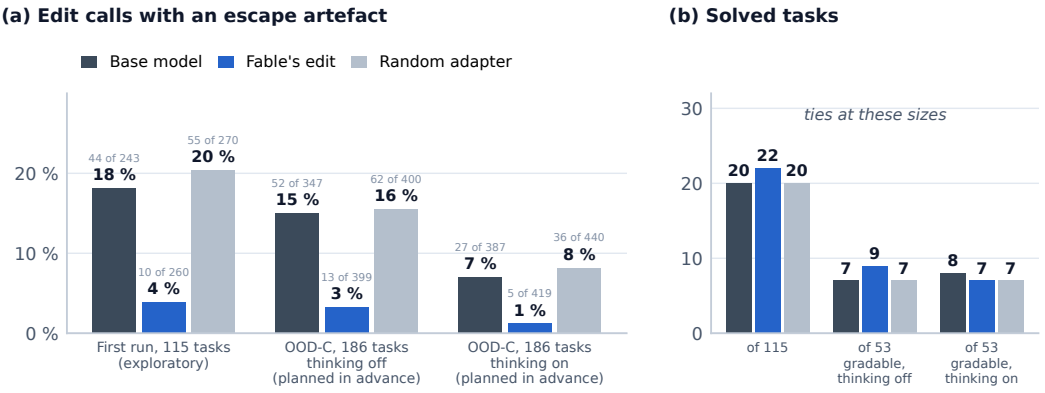


Figure 7: In live runs the selected edit cut escape artefacts with thinking off and on, and no difference in solved tasks was detected. Shares pool counted calls; the tests compare paired tasks.

Table 4: Live agent runs: one run per arm, served as a LoRA adapter on the 4-bit model in vLLM.

Task set	Arm	Edit calls with an artefact ↓	Old text not found	Accepted only through the fallback	Solved tasks
First run: 115 tasks, OOD-43 and OOD-B (exploratory)	Base	18 % (44 of 243)	38 %	19 %	20 of 115
	Fable's edit	4 % (10 of 260)	35 %	6 %	22 of 115
	Random adapter	20 % (55 of 270)	34 %	24 %	20 of 115
OOD-C, 186 tasks, thinking off (planned in advance)	Base	15 % (52 of 347)	27 %	15 %	7 of 53
	Fable's edit	3 % (13 of 399)	34 %	5 %	9 of 53
	Random adapter	16 % (62 of 400)	32 %	14 %	7 of 53
OOD-C, 186 tasks, thinking on (planned in advance)	Base	7 % (27 of 387)	25 %	9 %	8 of 53
	Fable's edit	1 % (5 of 419)	35 %	3 %	7 of 53
	Random adapter	8 % (36 of 440)	36 %	8 %	7 of 53

3.4 Task outcomes and stuck retries

We detected no difference in solved tasks (Table 4). Comparisons this small cannot show equivalence. On OOD-C, calls whose old text was not found rose with both adapters, the random one included, so the rise is not specific to the edit's direction; its cause is untested.

Most of the base model's misses were retries: 131 of 202 came after an earlier miss in the same task, and 55 % of retries repeated, byte for byte, an old text that had already failed. Before planning a third round aimed at them, we fixed two go/no-go checks in advance. Sampling each of 71 held-out retries 8 times showed two kinds (Figure 8a): 35 repeated a failed old text in every draw and 32 never did, and sampling almost never moved a *stuck* call off its failed text. A linear probe told retries from first attempts well (AUC 0.96), so the state is readable. But even with every failed old text deleted from the model's own earlier turns, stuck calls repeated it in 48 % of draws and only 17 % were accepted, far below the 40 % bar we had set (Figure 8b). Both checks failed, so we did not run the third round.

3.5 Round 1 and a tool-side change

A first round aimed at a broader target, whether the old text is found at all. Opus and Fable each wrote one edit with 10 forward-pass requests, and a recipe of ours a third. On 225 held-back calls from the training repositories none helped (264, 264 and 262 accepted of 450 generations, against 264 for the base), and size-matched random edits moved acceptance as much. The recipe removed a direction that predicted the failure (AUC 0.88), yet nothing changed; a probe's success does not show that the model uses what it reads [11]. Independent checks reproduced parts of the editors' explanations of their edits, but the effects were far too small to change what the model wrote.

A tool-side change did what the weight edits did not. In a test planned in advance on the same calls, showing tool results in the chat template's native form instead of an escaped string raised acceptance by 2.0 points ($p = 0.031$). The effect is small and narrow: the change bundles escaping, field layout and slightly shorter prompts, and the calls had already served round 1's test.

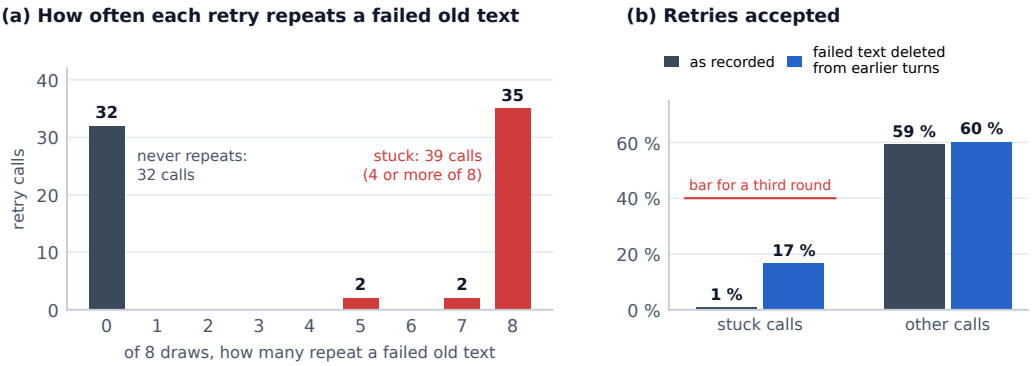


Figure 8: (a) Retries either always repeat a failed old text or never do. (b) Even with the failed text deleted from the context, few stuck retries are accepted; non-stuck acceptance stays near 60 %.

Round 1's edits left escape artefacts unchanged; that, with the artefact count in the base model's held-out calls, led to the narrower target of round 2.

We later ran the same tool-side change on the round-2 gate calls, registered before any generation; a tool-side change has no size-matched random control. It cut escape artefacts from 100 to 78 of 640 (fewer in 12 tasks, more in none), close to Opus's edit (77). Under the competition's harness acceptance did not move (+0.1 points; better in 4 tasks, worse in 4), and 1 and 2 working calls were lost per draw, at the limit of 2. Re-scored without the unescape fallback, acceptance rose by 3.7 points (better in 12 tasks, worse in 1) and none of the 177 and 174 working calls was lost. Of the escaped copies the fallback had rescued, the change moved 21 off the fallback and 4 of those failed, against 24 and 5 for Opus's edit. In the replay that regenerates both texts, the change left 56 of the base's 89 escaped old texts and 39 of its 67 escaped new texts (Opus's edit 65 and 47, Fable's 34 and 17), short of the halving we had set beforehand as the mark for the escaped view being their main source. There it also missed the compile guard under both rules: 10.3 % of accepted calls did not compile, against the base's 7.9 % and a limit of 9.9 %, and 9.0 % against 5.5 % and 7.5 % without the fallback; Opus's and Fable's edits met this guard under both. The excess sits in one repository: 10 of the 21 new compile failures fall on its two tasks, and without it the change meets the guard under both rules. So the dependence on the matching rule is not specific to weight edits, the round-1 gain of 2.0 points did not recur on this set, and the tool-side change also failed the round-2 guards, on the compile guard alone.

4 Related work and scope

Weight-space programming showed that frontier models can write changes to network weights directly, on small networks and a 0.8-billion-parameter model [1]; we add a deployed coding agent, held-out repositories, live runs and a gradient baseline. ROME writes a human-designed rank-one update where a fact is stored [6], and activation steering changes intermediate activations at inference time [7]. Left to themselves, the editors rebuilt known families of edits, but none tried updates that restrict or penalise changes on preserved calls' activations, such as a null-space constraint [9] or a covariance-weighted key-value edit [10]; with preservation as the binding constraint, these are the obvious next test, though they bound activations, not outputs. On the loop failure of [1], a linear program that bounds the edit's effect on preserved tokens matched the frontier model's edit with 8× fewer changed weights, and a steering vector did as well; because a constant push sufficed there, its author names a behaviour that changes some tokens and leaves others alone as the better test [13]. Escape artefacts are such a behaviour: escapes must leave the old text but stay where the file has them, and one editor's candidate broke legitimate escapes in new code. Agent success also depends on its tool interface [8], as our tool-side results in Section 3.5 show.

The study covers one model, one main target, one run per editor, one live run per arm in each setting, and one training seed. The companion writeup adds a second target: three edits cut E4B's wrong-version pydantic calls on held-out cases from 0.29 to at most 0.04, also failing a preservation check. We chose the target after counting artefacts in the base model's test calls, and the deployed edit after the replay; the replications fix the edit, not those choices. Missing traces and file states reduce the paired samples, and no zero-weight adapter separates serving effects from perturbation effects.

The lesson that carries forward: a tool's tolerance decides what a behaviour fix is worth. Where the tool already repairs a malformed argument, removing the malformation gains nothing on those calls and can lose some; without the repair, the same edits gain. Preservation must therefore be tested apart from the target change, and live behaviour apart from both. Before spending an editor's budget, it pays to measure how much an ideal fix of the target could gain, as the retry checks did.

5 Resources

`weightprog` (Apache-2.0, alpha) packages edit constructions, model instruments, size-matched random controls and comparisons, with MLX and PyTorch backends, for editing Gemma 4 toward a new target from a user's own coding agent. A companion writeup describes it and the data below. On the real E4B its base generation, backend agreement and captured edit effects match the study's outputs; edited generation has run through its MLX backend, and this study's adapters came from its export helper. OOD-C packages the 186 tasks for offline agent runs (SWE-rebench sources, CC-BY-4.0; open-source repositories only; 14.4 GB with snapshots and wheels), with gradability flags. The replay set holds the 320 held-out calls with their prompts and file states. The repository also holds the plans, logs, frozen reports, editor records and a long-form record of every test. Library: github.com/ozgung/weightprog; OOD-C: kaggle.com/datasets/ozgung/g4-oodc; replay set: kaggle.com/datasets/ozgung/g4-weight-edit-replay; repository: github.com/ozgung/gemma4-weight-edit-study.

6 Conclusion

Frontier models wrote gradient-free weight edits that reduced a specific coding-agent behaviour beyond matched controls, and one edit kept that change in the live agent, with thinking off and on. No edit or gradient-trained adapter met our preservation rule under the competition's harness, and no task benefit was detected. Re-scored post hoc without the harness's repair, two edits met the guards and raised acceptance: the measured cost depends on the matching rule as well as the edit.

References

1. Takehi, Teoh, Zhang, Khattab. Sparks of Weight-Space Programming. Blog post, 2026.
2. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.
3. van Miltenburg, van der Lee, Krahmer. Preregistering NLP Research. NAACL 2021.
4. Badertdinov et al. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. NeurIPS 2025.
5. Badertdinov, Nekrashevich, Shevtsov, Golubev. SWE-rebench V2: Language-Agnostic SWE Task Collection at Scale. 2026.
6. Meng, Bau, Andonian, Belinkov. Locating and Editing Factual Associations in GPT. NeurIPS 2022.
7. Turner et al. Steering Language Models With Activation Engineering. arXiv 2023.
8. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024.
9. Fang et al. AlphaEdit: Null-Space Constrained Knowledge Editing for Language Models. ICLR 2025.
10. Meng, Sharma, Andonian, Belinkov, Bau. Mass-Editing Memory in a Transformer. ICLR 2023.
11. Hewitt, Liang. Designing and Interpreting Probes with Control Tasks. EMNLP 2019.
12. Yan et al. Positive-Congruent Training: Towards Regression-Free Model Updates. CVPR 2021.
13. Barry. Can a Linear Program Do Weight-Space Programming?. Blog post, 2026.